

Chapter 5: AI/ML Integration & Advanced APIs

Welcome to the **cutting-edge level**! In this chapter, we'll integrate AI/ML capabilities, implement GraphQL APIs, and build advanced search and analytics systems.

Chapter 5 Learning Goals

By the end of this chapter, you'll master:

- **AI/ML Integration** (OpenAI, TensorFlow, custom models)
- **GraphQL APIs** (Schema design, resolvers, subscriptions)
- **Advanced Search** (Elasticsearch, fuzzy search, recommendations)
- **Analytics & Big Data** (Data processing, insights, reporting)
- **Third-party Integrations** (Payment gateways, social auth, APIs)
- **Advanced File Processing** (Image recognition, video processing)
- **Real-time Analytics** (Streaming data, live dashboards)

AI/ML Integration

1. OpenAI Integration Setup

```
npm install openai tiktoken sharp multer-s3 aws-sdk
```

[Copy](#)

```
// services/ai-service/src/config/openai.js
const { OpenAI } = require('openai');
const logger = require('../../../shared/utils/logger');

class AIService {
  constructor() {
    this.openai = new OpenAI({
      apiKey: process.env.OPENAI_API_KEY,
      timeout: 30000,
      maxRetries: 3
    });
  }

  this.models = {
    gpt4: 'gpt-4-turbo-preview',
    gpt35: 'gpt-3.5-turbo',
    embedding: 'text-embedding-3-small',
    dalle: 'dall-e-3',
  }
}
```

[Copy](#)

```
    whisper: 'whisper-1'
  };
}

// Text generation and completion
async generateText(prompt, options = {}) {
  try {
    const response = await this.openai.chat.completions.create({
      model: options.model || this.models.gpt35,
      messages: [
        {
          role: 'system',
          content: options.systemPrompt || 'You are a helpful assistant.'
        },
        {
          role: 'user',
          content: prompt
        }
      ],
      max_tokens: options.maxTokens || 1000,
      temperature: options.temperature || 0.7,
      top_p: options.topP || 1,
      frequency_penalty: options.frequencyPenalty || 0,
      presence_penalty: options.presencePenalty || 0
    });

    const result = {
      text: response.choices[0].message.content,
      usage: response.usage,
      model: response.model,
      finishReason: response.choices[0].finish_reason
    };

    logger.info('Text generation completed:', {
      model: result.model,
      tokens: result.usage.total_tokens,
      finishReason: result.finishReason
    });

    return result;
  } catch (error) {
    logger.error('Text generation failed:', error);
    throw new Error(`AI text generation failed: ${error.message}`);
  }
}

// Content moderation
async moderateContent(text) {
  try {
```

```
const response = await this.openai.moderations.create({
  input: text
});

const result = response.results[0];

return {
  flagged: result.flagged,
  categories: result.categories,
  categoryScores: result.category_scores,
  safe: !result.flagged
};
} catch (error) {
  logger.error('Content moderation failed:', error);
  throw new Error(`Content moderation failed: ${error.message}`);
}
}

// Generate embeddings for semantic search
async generateEmbeddings(texts) {
  try {
    const response = await this.openai.embeddings.create({
      model: this.models.embedding,
      input: Array.isArray(texts) ? texts : [texts]
    });

    const embeddings = response.data.map(item => ({
      embedding: item.embedding,
      index: item.index
    }));

    return {
      embeddings,
      usage: response.usage,
      model: response.model
    };
  } catch (error) {
    logger.error('Embedding generation failed:', error);
    throw new Error(`Embedding generation failed: ${error.message}`);
  }
}

// Image generation with DALL-E
async generateImage(prompt, options = {}) {
  try {
    const response = await this.openai.images.generate({
      model: this.models.dalle,
      prompt,
      n: options.count || 1,

```

```

        size: options.size || '1024x1024',
        quality: options.quality || 'standard',
        style: options.style || 'vivid',
        response_format: 'url'
    });

    return {
        images: response.data.map(image => ({
            url: image.url,
            revisedPrompt: image.revised_prompt
        })),
        prompt,
        revisedPrompt: response.data[0].revised_prompt
    };
} catch (error) {
    logger.error('Image generation failed:', error);
    throw new Error(`Image generation failed: ${error.message}`);
}

}

// Speech to text with Whisper
async transcribeAudio(audioBuffer, options = {}) {
    try {
        const response = await this.openai.audio.transcriptions.create({
            file: audioBuffer,
            model: this.models.whisper,
            language: options.language,
            response_format: options.format || 'json',
            temperature: options.temperature || 0
        });

        return {
            text: response.text,
            language: response.language,
            duration: response.duration,
            segments: response.segments
        };
    } catch (error) {
        logger.error('Audio transcription failed:', error);
        throw new Error(`Audio transcription failed: ${error.message}`);
    }
}

// Chat completion with conversation history
async chatCompletion(messages, options = {}) {
    try {
        const response = await this.openai.chat.completions.create({
            model: options.model || this.models.gpt35,
            messages,

```

```

    max_tokens: options.maxTokens || 1000,
    temperature: options.temperature || 0.7,
    stream: options.stream || false
  });

  if (options.stream) {
    return response; // Return stream for real-time responses
  }

  return {
    message: response.choices[0].message,
    usage: response.usage,
    finishReason: response.choices[0].finish_reason
  };
} catch (error) {
  logger.error('Chat completion failed:', error);
  throw new Error(`Chat completion failed: ${error.message}`);
}
}
}

module.exports = new AIService();

```

2. AI-Powered Content Generation Service

```

// services/ai-service/src/services/content-generator.service.js
const aiService = require('../config/openai');
const logger = require('../../../../shared/utils/logger');

class ContentGeneratorService {

  // Generate blog post from topic
  async generateBlogPost(topic, options = {}) {
    const systemPrompt = `You are a professional content writer. Create engaging,
    well-structured blog posts that are informative and SEO-friendly. Include proper
    headings, subheadings, and maintain a consistent tone throughout.`;

    const prompt = `Write a comprehensive blog post about "${topic}".
    Requirements:
    - Length: ${options.length || 'medium'} (400-800 words)
    - Tone: ${options.tone || 'professional but engaging'}
    - Target audience: ${options.audience || 'general readers'}
    - Include: Introduction, main content with subheadings, conclusion
    - Add relevant keywords for SEO
    ${options.includeCallToAction ? '- Include a call-to-action at the end' : ''}

    Format the response as JSON with the following structure:

```

 Copy

```

{
  "title": "Blog post title",
  "excerpt": "Brief summary for preview",
  "content": "Full blog post content with markdown formatting",
  "tags": ["relevant", "tags"],
  "metaDescription": "SEO meta description",
  "readTime": estimated_read_time_in_minutes
}`;

try {
  const result = await aiService.generateText(prompt, {
    model: 'gpt-4-turbo-preview',
    systemPrompt,
    maxTokens: 2000,
    temperature: 0.8
  });

  const blogPost = JSON.parse(result.text);

  return {
    ...blogPost,
    aiGenerated: true,
    generatedAt: new Date(),
    usage: result.usage
  };
} catch (error) {
  logger.error('Blog post generation failed:', error);
  throw new Error('Failed to generate blog post');
}
}

// Generate social media posts
async generateSocialPosts(content, platforms = ['twitter', 'facebook',
'linkedin']) {
  const posts = {};

  for (const platform of platforms) {
    const prompt = this.getSocialPrompt(platform, content);

    try {
      const result = await aiService.generateText(prompt, {
        maxTokens: 300,
        temperature: 0.9
      });

      posts[platform] = {
        content: result.text.trim(),
        characterCount: result.text.trim().length,
        hashtags: this.extractHashtags(result.text),

```

```

    platform
  };
} catch (error) {
  logger.error(`Social post generation failed for ${platform}:`, error);
  posts[platform] = { error: error.message };
}
}

return posts;
}

getSocialPrompt(platform, content) {
  const prompts = {
    twitter: `Create an engaging Twitter post based on this content:
"${content}".
    Requirements: Max 280 characters, include relevant hashtags, engaging and
concise.`,

    facebook: `Create a Facebook post based on this content: "${content}".
    Requirements: Engaging tone, 1-2 paragraphs, include call-to-action,
relevant hashtags.`,

    linkedin: `Create a professional LinkedIn post based on this content:
"${content}".
    Requirements: Professional tone, thought-provoking, include industry
insights, relevant hashtags.`,

    instagram: `Create an Instagram caption based on this content: "${content}".
    Requirements: Visual-focused, engaging, include story elements, relevant
hashtags and emojis.`
  };

  return prompts[platform] || prompts.twitter;
}

extractHashtags(text) {
  const hashtagRegex = /#[\w]+/g;
  return text.match(hashtagRegex) || [];
}

// Generate product descriptions
async generateProductDescription(productInfo) {
  const prompt = `Create a compelling product description for an e-commerce
website.

Product Information:
- Name: ${productInfo.name}
- Category: ${productInfo.category}
- Key Features: ${productInfo.features?.join(', ') || 'N/A'}`

```

- Target Audience: `${productInfo.targetAudience || 'General consumers'}`
- Price Range: `${productInfo.priceRange || 'Not specified'}`

Generate a description that:

1. Highlights key benefits and features
2. Uses persuasive language
3. Includes SEO-friendly keywords
4. Is scannable with bullet points
5. Includes a strong call-to-action

Format as JSON:

```
{
  "shortDescription": "Brief 1-2 sentence overview",
  "fullDescription": "Detailed description with formatting",
  "bulletPoints": ["key", "features", "list"],
  "seoKeywords": ["relevant", "keywords"],
  "callToAction": "Compelling CTA text"
};
```

```
try {
  const result = await aiService.generateText(prompt, {
    maxTokens: 1000,
    temperature: 0.7
  });

  return JSON.parse(result.text);
} catch (error) {
  logger.error('Product description generation failed:', error);
  throw new Error('Failed to generate product description');
}

// Content summarization
async summarizeContent(content, options = {}) {
  const prompt = `Summarize the following content in ${options.length || 'medium'} length (${options.maxWords || 100} words max):

  ${content}

  Requirements:
  - Capture main points and key insights
  - Maintain original tone and meaning
  - Be concise and clear
  - ${options.bulletPoints ? 'Format as bullet points' : 'Use paragraph format'}`;

  try {
    const result = await aiService.generateText(prompt, {
      maxTokens: options.maxTokens || 300,
```

```

        temperature: 0.3
    });

    return {
        summary: result.text.trim(),
        originalLength: content.length,
        summaryLength: result.text.trim().length,
        compressionRatio: (result.text.trim().length / content.length *
100).toFixed(1) + '%'
    };
} catch (error) {
    logger.error('Content summarization failed:', error);
    throw new Error('Failed to summarize content');
}
}

// Generate SEO metadata
async generateSEOMetadata(content, targetKeywords = []) {
    const prompt = `Generate SEO metadata for the following content:

    ${content.substring(0, 1000)}...

    Target Keywords: ${targetKeywords.join(', ')}

    Generate:
    1. SEO-optimized title (50-60 characters)
    2. Meta description (150-160 characters)
    3. H1 tag suggestion
    4. Additional relevant keywords
    5. Suggested internal link opportunities

    Format as JSON:
    {
        "title": "SEO title",
        "metaDescription": "Meta description",
        "h1": "H1 tag",
        "keywords": ["additional", "keywords"],
        "internalLinks": ["suggested", "internal", "links"],
        "focusKeyword": "primary keyword"
    }`;

    try {
        const result = await aiService.generateText(prompt, {
            maxTokens: 500,
            temperature: 0.5
        });

        return JSON.parse(result.text);
    } catch (error) {

```

```

    logger.error('SEO metadata generation failed:', error);
    throw new Error('Failed to generate SEO metadata');
  }
}
}

module.exports = new ContentGeneratorService();

```

3. AI-Powered Content Moderation

 Copy

```

// services/ai-service/src/services/content-moderation.service.js
const aiService = require('../config/openai');
const logger = require('../../../../shared/utils/logger');

class ContentModerationService {

  constructor() {
    this.toxicityThreshold = 0.7;
    this.spamThreshold = 0.8;
    this.bannedWords = new Set([
      // Add your banned words list
    ]);
  }

  // Comprehensive content analysis
  async analyzeContent(content, options = {}) {
    try {
      const analyses = await Promise.allSettled([
        this.checkToxicity(content),
        this.checkSpam(content),
        this.checkProfanity(content),
        this.analyzeSentiment(content),
        this.detectLanguage(content),
        this.checkReadability(content)
      ]);

      const [toxicity, spam, profanity, sentiment, language, readability] =
        analyses.map(
          result => result.status === 'fulfilled' ? result.value : { error:
            result.reason.message }
        );

      // Calculate overall safety score
      const safetyScore = this.calculateSafetyScore({
        toxicity: toxicity.safe,
        spam: spam.safe,
        profanity: profanity.safe

```

```
});

const analysis = {
  content: content.substring(0, 100) + '...',
  timestamp: new Date().toISOString(),
  safetyScore,
  approved: safetyScore >= 0.7,
  analyses: {
    toxicity,
    spam,
    profanity,
    sentiment,
    language,
    readability
  },
  recommendations: this.generateRecommendations({
    toxicity,
    spam,
    profanity,
    sentiment,
    readability
  })
};

// Log concerning content
if (analysis.safetyScore < 0.5) {
  logger.warn('Content flagged for review:', {
    safetyScore: analysis.safetyScore,
    userId: options.userId,
    contentType: options.contentType
  });
}

return analysis;
} catch (error) {
  logger.error('Content analysis failed:', error);
  throw new Error('Content analysis failed');
}
}

// Check for toxic content using OpenAI moderation
async checkToxicity(content) {
  try {
    const moderation = await aiService.moderateContent(content);

    const toxicCategories = Object.entries(moderation.categories)
      .filter(([, flagged]) => flagged)
      .map(([category]) => category);
  }
}
```

```
const highestScore = Math.max(...Object.values(moderation.categoryScores));

return {
  safe: !moderation.flagged,
  score: 1 - highestScore, // Convert to safety score
  flaggedCategories: toxicCategories,
  confidence: highestScore,
  details: {
    categories: moderation.categories,
    scores: moderation.categoryScores
  }
};
} catch (error) {
  logger.error('Toxicity check failed:', error);
  return { safe: true, score: 0.5, error: error.message };
}
}

// AI-powered spam detection
async checkSpam(content) {
  const prompt = `Analyze the following content for spam characteristics:

"${content}"

Check for:
1. Excessive promotional language
2. Repetitive phrases
3. Suspicious links or offers
4. Generic/automated content
5. Misleading claims

Respond with JSON:
{
  "isSpam": boolean,
  "confidence": number_between_0_and_1,
  "reasons": ["list", "of", "spam", "indicators"],
  "spamScore": number_between_0_and_1
}`;

  try {
    const result = await aiService.generateText(prompt, {
      maxTokens: 300,
      temperature: 0.2
    });

    const analysis = JSON.parse(result.text);

    return {
      safe: !analysis.isSpam,
```

```

    score: 1 - analysis.spamScore,
    confidence: analysis.confidence,
    reasons: analysis.reasons,
    spamScore: analysis.spamScore
  };
} catch (error) {
  logger.error('Spam check failed:', error);
  return { safe: true, score: 0.5, error: error.message };
}
}

// Profanity and inappropriate content detection
async checkProfanity(content) {
  try {
    // Simple word-based check
    const words = content.toLowerCase().split(/\s+/);
    const bannedWordsFound = words.filter(word => this.bannedWords.has(word));

    // AI-powered context analysis
    const prompt = `Check if the following content contains inappropriate
language, considering context and intent:

"${content}"

Look for:
1. Profanity and vulgar language
2. Hate speech
3. Discriminatory language
4. Inappropriate sexual content

Respond with JSON:
{
  "hasInappropriateLang": boolean,
  "severity": "low|medium|high",
  "context": "brief explanation",
  "score": number_between_0_and_1
}`;

    const result = await aiService.generateText(prompt, {
      maxTokens: 200,
      temperature: 0.1
    });

    const analysis = JSON.parse(result.text);

    return {
      safe: !analysis.hasInappropriateLang && bannedWordsFound.length === 0,
      score: 1 - analysis.score,
      severity: analysis.severity,

```

```
        bannedWords: bannedWordsFound,
        context: analysis.context,
        confidence: analysis.score
    };
} catch (error) {
    logger.error('Profanity check failed:', error);
    return { safe: true, score: 0.5, error: error.message };
}
}

// Sentiment analysis
async analyzeSentiment(content) {
    const prompt = `Analyze the sentiment and emotional tone of this content:

"${content}"

Provide:
1. Overall sentiment (positive/negative/neutral)
2. Emotional tone
3. Confidence level
4. Key emotional indicators

Respond with JSON:
{
    "sentiment": "positive|negative|neutral",
    "score": number_between_-1_and_1,
    "emotions": ["joy", "anger", "fear", "etc"],
    "confidence": number_between_0_and_1,
    "tone": "description of tone"
}`;

    try {
        const result = await aiService.generateText(prompt, {
            maxTokens: 300,
            temperature: 0.3
        });

        return JSON.parse(result.text);
    } catch (error) {
        logger.error('Sentiment analysis failed:', error);
        return { sentiment: 'neutral', score: 0, error: error.message };
    }
}

// Language detection
async detectLanguage(content) {
    const prompt = `Detect the primary language of this text and estimate
confidence:
```

```
"${content.substring(0, 200)}"
```

Respond with JSON:

```
{
  "language": "language_code",
  "languageName": "language_name",
  "confidence": number_between_0_and_1,
  "isEnglish": boolean
};
```

```
try {
  const result = await aiService.generateText(prompt, {
    maxTokens: 100,
    temperature: 0.1
  });

  return JSON.parse(result.text);
} catch (error) {
  logger.error('Language detection failed:', error);
  return { language: 'en', languageName: 'English', confidence: 0.5 };
}

// Content readability analysis
async checkReadability(content) {
  const words = content.split(/\s+/).length;
  const sentences = content.split(/[.!?]+/).length;
  const avgWordsPerSentence = words / sentences;

  // Simple readability metrics
  let readabilityScore = 100;
  if (avgWordsPerSentence > 20) readabilityScore -= 20;
  if (avgWordsPerSentence > 30) readabilityScore -= 30;

  const longWords = content.split(/\s+/).filter(word => word.length > 6).length;
  const longWordRatio = longWords / words;
  if (longWordRatio > 0.3) readabilityScore -= 20;

  return {
    score: Math.max(0, readabilityScore) / 100,
    wordsCount: words,
    sentencesCount: sentences,
    avgWordsPerSentence,
    longWordRatio,
    readingLevel: this.getReadingLevel(readabilityScore),
    suggestions: this.getReadabilitySuggestions(avgWordsPerSentence,
longWordRatio)
  };
}
```

```
getReadingLevel(score) {
  if (score >= 90) return 'Very Easy';
  if (score >= 80) return 'Easy';
  if (score >= 70) return 'Fairly Easy';
  if (score >= 60) return 'Standard';
  if (score >= 50) return 'Fairly Difficult';
  if (score >= 30) return 'Difficult';
  return 'Very Difficult';
}

getReadabilitySuggestions(avgWords, longWordRatio) {
  const suggestions = [];

  if (avgWords > 20) {
    suggestions.push('Consider breaking long sentences into shorter ones');
  }
  if (longWordRatio > 0.3) {
    suggestions.push('Try using simpler, more common words where possible');
  }
  if (suggestions.length === 0) {
    suggestions.push('Content readability is good');
  }

  return suggestions;
}

calculateSafetyScore(checks) {
  const weights = { toxicity: 0.4, spam: 0.3, profanity: 0.3 };
  let score = 0;

  Object.entries(weights).forEach(([check, weight]) => {
    score += checks[check] ? weight : 0;
  });

  return score;
}

generateRecommendations(analyses) {
  const recommendations = [];

  if (!analyses.toxicity?.safe) {
    recommendations.push('Content contains toxic language and should be reviewed');
  }
  if (!analyses.spam?.safe) {
    recommendations.push('Content appears to be spam or promotional');
  }
  if (!analyses.profanity?.safe) {
```

```

    recommendations.push('Content contains inappropriate language');
  }
  if (analyses.readability?.score < 0.5) {
    recommendations.push('Content could be more readable - consider simplifying language');
  }
  if (analyses.sentiment?.sentiment === 'negative' && analyses.sentiment?.score < -0.7) {
    recommendations.push('Content has very negative sentiment - consider review');
  }

  if (recommendations.length === 0) {
    recommendations.push('Content appears to be appropriate');
  }

  return recommendations;
}
}

module.exports = new ContentModerationService();

```

GraphQL API Implementation

1. GraphQL Server Setup

```

npm install apollo-server-express graphql graphql-tools graphql-upload
npm install dataloader graphql-depth-limit graphql-query-complexity

```

 Copy

```

// services/graphql-service/src/app.js
const express = require('express');
const { ApolloServer } = require('apollo-server-express');
const { makeExecutableSchema } = require('@graphql-tools/schema');
const { graphqlUploadExpress } = require('graphql-upload');
const depthLimit = require('graphql-depth-limit');
const costAnalysis = require('graphql-query-complexity').costAnalysisMaximumRuleError;

const typeDefs = require('./schema');
const resolvers = require('./resolvers');
const { createDataLoaders } = require('./dataloaders');
const { authMiddleware } = require('./middleware/auth');
const logger = require('../../../../shared/utils/logger');

```

 Copy

```
class GraphQLService {
  constructor() {
    this.app = express();
    this.server = null;
  }

  async initialize() {
    // Enable file uploads
    this.app.use(graphqlUploadExpress({ maxFileSize: 10000000, maxFiles: 10 }));

    // Create executable schema
    const schema = makeExecutableSchema({
      typeDefs,
      resolvers
    });

    // Create Apollo Server
    this.server = new ApolloServer({
      schema,
      context: async ({ req, connection }) => {
        // WebSocket connection (subscriptions)
        if (connection) {
          return {
            ...connection.context,
            dataLoaders: createDataLoaders()
          };
        }

        // HTTP request
        const user = await authMiddleware(req);

        return {
          req,
          user,
          dataLoaders: createDataLoaders(),
          logger
        };
      },
      validationRules: [
        depthLimit(10), // Limit query depth
        costAnalysis({
          maximumCost: 1000,
          defaultCost: 1,
          variables: {},
          createError: (max, actual) => {
            return new Error(`Query cost ${actual} exceeds maximum cost ${max}`);
          }
        })
      ]
    });
  }
}
```

```

    ],
    formatError: (error) => {
      logger.error('GraphQL Error:', {
        message: error.message,
        path: error.path,
        source: error.source
      });

      // Don't expose internal errors in production
      if (process.env.NODE_ENV === 'production' &&
error.message.includes('Database')) {
        return new Error('Internal server error');
      }

      return error;
    },
    introspection: process.env.NODE_ENV !== 'production',
    playground: process.env.NODE_ENV !== 'production',
    subscriptions: {
      path: '/graphql',
      onConnect: async (connectionParams, websocket, context) => {
        // Handle WebSocket authentication
        const token = connectionParams.authorization;
        if (token) {
          const user = await authMiddleware({ headers: { authorization: token }
});
          return { user };
        }
        return {};
      },
      onDisconnect: (websocket, context) => {
        logger.info('WebSocket disconnected');
      }
    }
  });

  await this.server.start();
  this.server.applyMiddleware({ app: this.app, path: '/graphql' });

  // Health check endpoint
  this.app.get('/health', (req, res) => {
    res.json({ status: 'OK', service: 'GraphQL' });
  });

  return this.app;
}

getServer() {
  return this.server;
}

```

```
}  
}  
  
module.exports = new GraphQLService();
```

2. GraphQL Schema Definition

```
// services/graphql-service/src/schema/index.js  
const { gql } = require('apollo-server-express');  
  
const typeDefs = gql`  
  # Scalars  
  scalar DateTime  
  scalar Upload  
  scalar JSON  
  
  # Enums  
  enum UserRole {  
    USER  
    ADMIN  
    MODERATOR  
  }  
  
  enum PostStatus {  
    DRAFT  
    PUBLISHED  
    ARCHIVED  
  }  
  
  enum OrderBy {  
    ASC  
    DESC  
  }  
  
  # Input Types  
  input UserInput {  
    name: String!  
    email: String!  
    password: String!  
  }  
  
  input PostInput {  
    title: String!  
    content: String!  
    tags: [String!]  
    categoryId: ID  
    featuredImage: Upload
```

 Copy

```
    status: PostStatus = DRAFT
  }

  input PostUpdateInput {
    title: String
    content: String
    tags: [String!]
    categoryId: ID
    status: PostStatus
  }

  input PostFilters {
    authorId: ID
    categoryId: ID
    tags: [String!]
    status: PostStatus
    search: String
    dateFrom: DateTime
    dateTo: DateTime
  }

  input PaginationInput {
    page: Int = 1
    limit: Int = 10
    orderBy: String = "createdAt"
    order: OrderBy = DESC
  }

# Types
type User {
  id: ID!
  name: String!
  email: String!
  role: UserRole!
  avatar: String
  bio: String
  website: String
  location: String
  verified: Boolean!
  followersCount: Int!
  followingCount: Int!
  postsCount: Int!
  posts(pagination: PaginationInput): PostConnection!
  followers(pagination: PaginationInput): UserConnection!
  following(pagination: PaginationInput): UserConnection!
  isFollowing: Boolean
  createdAt: DateTime!
  updatedAt: DateTime!
}
```

```
type Post {
  id: ID!
  title: String!
  content: String!
  excerpt: String
  slug: String!
  tags: [String!]!
  category: Category
  author: User!
  featuredImage: String
  status: PostStatus!
  likesCount: Int!
  commentsCount: Int!
  viewsCount: Int!
  readTime: Int!
  likes: [Like!]!
  comments(pagination: PaginationInput): CommentConnection!
  isLiked: Boolean
  publishedAt: DateTime
  createdAt: DateTime!
  updatedAt: DateTime!
}

type Category {
  id: ID!
  name: String!
  slug: String!
  description: String
  postsCount: Int!
  posts(pagination: PaginationInput): PostConnection!
  createdAt: DateTime!
}

type Comment {
  id: ID!
  content: String!
  author: User!
  post: Post!
  parent: Comment
  replies: [Comment!]!
  likesCount: Int!
  isLiked: Boolean
  createdAt: DateTime!
  updatedAt: DateTime!
}

type Like {
  id: ID!
```

```
    user: User!
    post: Post
    comment: Comment
    createdAt: DateTime!
  }

  type Notification {
    id: ID!
    recipient: User!
    sender: User
    type: NotificationType!
    message: String!
    data: JSON
    read: Boolean!
    createdAt: DateTime!
  }

  enum NotificationType {
    LIKE
    COMMENT
    FOLLOW
    POST
    SYSTEM
  }

  # Connection Types (for pagination)
  type PostConnection {
    edges: [PostEdge!]!
    pageInfo: PageInfo!
    totalCount: Int!
  }

  type PostEdge {
    node: Post!
    cursor: String!
  }

  type UserConnection {
    edges: [UserEdge!]!
    pageInfo: PageInfo!
    totalCount: Int!
  }

  type UserEdge {
    node: User!
    cursor: String!
  }

  type CommentConnection {
```

```
    edges: [CommentEdge!]!
    pageInfo: PageInfo!
    totalCount: Int!
}

type CommentEdge {
  node: Comment!
  cursor: String!
}

type PageInfo {
  hasNextPage: Boolean!
  hasPreviousPage: Boolean!
  startCursor: String
  endCursor: String
}

# Authentication Types
type AuthPayload {
  token: String!
  user: User!
  expiresAt: DateTime!
}

# AI Generated Content Types
type GeneratedContent {
  title: String!
  content: String!
  excerpt: String!
  tags: [String!]!
  metaDescription: String!
  readTime: Int!
  aiGenerated: Boolean!
}

type ContentAnalysis {
  safetyScore: Float!
  approved: Boolean!
  sentiment: String!
  language: String!
  readabilityScore: Float!
  recommendations: [String!]!
}

# Analytics Types
type PostAnalytics {
  viewsCount: Int!
  likesCount: Int!
  commentsCount: Int!
```

```
    sharesCount: Int!
    engagementRate: Float!
    topReferrers: [String!]!
    viewsByDate: [DateCount!]!
}

type DateCount {
    date: DateTime!
    count: Int!
}

type UserAnalytics {
    totalPosts: Int!
    totalLikes: Int!
    totalViews: Int!
    followersGrowth: [DateCount!]!
    topPosts: [Post!]!
    engagementStats: EngagementStats!
}

type EngagementStats {
    averageLikes: Float!
    averageComments: Float!
    averageViews: Float!
    engagementRate: Float!
}

# Root Types
type Query {
    # User queries
    me: User
    user(id: ID!): User
    users(pagination: PaginationInput, search: String): UserConnection!

    # Post queries
    post(id: ID, slug: String): Post
    posts(filters: PostFilters, pagination: PaginationInput): PostConnection!
    trendingPosts(limit: Int = 10): [Post!]!

    # Category queries
    categories: [Category!]!
    category(id: ID, slug: String): Category

    # Comment queries
    comment(id: ID!): Comment

    # Notification queries
    notifications(pagination: PaginationInput): [Notification!]!
    unreadNotificationsCount: Int!
```

```
# Search queries
search(query: String!, type: String, pagination: PaginationInput):
SearchResults!

# Analytics queries
postAnalytics(postId: ID!): PostAnalytics!
userAnalytics(userId: ID): UserAnalytics!
dashboardStats: DashboardStats!
}

type Mutation {
  # Authentication
  register(input: UserInput!): AuthPayload!
  login(email: String!, password: String!): AuthPayload!
  logout: Boolean!

  # User mutations
  updateProfile(input: UserUpdateInput!): User!
  uploadAvatar(file: Upload!): String!
  followUser(userId: ID!): Boolean!
  unfollowUser(userId: ID!): Boolean!

  # Post mutations
  createPost(input: PostInput!): Post!
  updatePost(id: ID!, input: PostUpdateInput!): Post!
  deletePost(id: ID!): Boolean!
  likePost(postId: ID!): Boolean!
  unlikePost(postId: ID!): Boolean!

  # AI-powered mutations
  generateBlogPost(topic: String!, options: GenerationOptions):
GeneratedContent!
  analyzeContent(content: String!): ContentAnalysis!
  generateSEO(content: String!): SEOMetadata!

  # Comment mutations
  addComment(postId: ID!, content: String!, parentId: ID): Comment!
  updateComment(id: ID!, content: String!): Comment!
  deleteComment(id: ID!): Boolean!
  likeComment(commentId: ID!): Boolean!

  # Notification mutations
  markNotificationRead(id: ID!): Boolean!
  markAllNotificationsRead: Boolean!
}

type Subscription {
  # Real-time updates
```

```
    postLiked(postId: ID!): Post!
    commentAdded(postId: ID!): Comment!
    notificationReceived: Notification!
    userOnline(userId: ID!): User!

    # Live analytics
    postAnalyticsUpdate(postId: ID!): PostAnalytics!
  }

# Additional types for various features
input GenerationOptions {
  length: String = "medium"
  tone: String = "professional"
  audience: String = "general"
  includeCallToAction: Boolean = false
}

input UserUpdateInput {
  name: String
  bio: String
  website: String
  location: String
}

type SEOMetadata {
  title: String!
  metaDescription: String!
  keywords: [String!]!
  focusKeyword: String!
}

union SearchResult = Post | User | Category

type SearchResults {
  results: [SearchResult!]!
  totalCount: Int!
  hasMore: Boolean!
}

type DashboardStats {
  totalUsers: Int!
  totalPosts: Int!
  totalComments: Int!
  totalLikes: Int!
  newUsersToday: Int!
  newPostsToday: Int!
  engagementRate: Float!
  topAuthors: [User!]!
  trendingTags: [String!]!
```

```

    }
  `;

module.exports = typeDefs;

```

3. GraphQL Resolvers

```

// services/graphql-service/src/resolvers/index.js
const { GraphQLUpload } = require('graphql-upload');
const { GraphQLDateTime, GraphQLJSON } = require('graphql-scalars');
const { PubSub, withFilter } = require('apollo-server-express');

const userResolvers = require('./user.resolvers');
const postResolvers = require('./post.resolvers');
const commentResolvers = require('./comment.resolvers');
const analyticsResolvers = require('./analytics.resolvers');
const aiResolvers = require('./ai.resolvers');

const pubsub = new PubSub();

const resolvers = {
  // Scalars
  Upload: GraphQLUpload,
  DateTime: GraphQLDateTime,
  JSON: GraphQLJSON,

  // Union resolvers
  SearchResult: {
    __resolveType(obj) {
      if (obj.title && obj.content) return 'Post';
      if (obj.name && obj.email) return 'User';
      if (obj.name && obj.slug && obj.description) return 'Category';
      return null;
    }
  },

  // Root resolvers
  Query: {
    // User queries
    me: (parent, args, { user }) => {
      if (!user) throw new Error('Not authenticated');
      return user;
    },

    user: async (parent, { id }, { dataloaders }) => {
      return await dataloaders.userLoader.load(id);
    },
  },

```

 Copy

```
users: async (parent, { pagination, search }, { dataloaders }) => {
  return await userResolvers.getUsers(pagination, search, dataloaders);
},

// Post queries
post: async (parent, { id, slug }, { dataloaders }) => {
  if (id) return await dataloaders.postLoader.load(id);
  if (slug) return await postResolvers.getPostBySlug(slug, dataloaders);
  throw new Error('Either id or slug must be provided');
},

posts: async (parent, { filters, pagination }, { dataloaders }) => {
  return await postResolvers.getPosts(filters, pagination, dataloaders);
},

trendingPosts: async (parent, { limit }, { dataloaders }) => {
  return await postResolvers.getTrendingPosts(limit, dataloaders);
},

// Search
search: async (parent, { query, type, pagination }, { dataloaders }) => {
  return await searchResolvers.search(query, type, pagination, dataloaders);
},

// Analytics
postAnalytics: async (parent, { postId }, { user, dataloaders }) => {
  return await analyticsResolvers.getPostAnalytics(postId, user, dataloaders);
},

userAnalytics: async (parent, { userId }, { user, dataloaders }) => {
  const targetUserId = userId || user.id;
  return await analyticsResolvers.getUserAnalytics(targetUserId, user,
dataaloaders);
},

dashboardStats: async (parent, args, { user, dataloaders }) => {
  if (user.role !== 'ADMIN') throw new Error('Admin access required');
  return await analyticsResolvers.getDashboardStats(dataloaders);
}
},

Mutation: {
  // Authentication
  register: async (parent, { input }) => {
    return await userResolvers.register(input);
  },

  login: async (parent, { email, password }) => {
```

```

    return await userResolvers.login(email, password);
  },

  // Posts
  createPost: async (parent, { input }, { user, dataloaders }) => {
    if (!user) throw new Error('Authentication required');
    const post = await postResolvers.createPost(input, user, dataloaders);

    // Publish to subscribers
    pubsub.publish('POST_CREATED', { postCreated: post });

    return post;
  },

  likePost: async (parent, { postId }, { user, dataloaders }) => {
    if (!user) throw new Error('Authentication required');

    const result = await postResolvers.likePost(postId, user, dataloaders);
    const post = await dataloaders.postLoader.load(postId);

    // Publish real-time update
    pubsub.publish('POST_LIKED', {
      postLiked: post,
      postId
    });

    return result;
  },

  // AI-powered mutations
  generateBlogPost: async (parent, { topic, options }, { user }) => {
    if (!user) throw new Error('Authentication required');
    return await aiResolvers.generateBlogPost(topic, options);
  },

  analyzeContent: async (parent, { content }, { user }) => {
    if (!user) throw new Error('Authentication required');
    return await aiResolvers.analyzeContent(content);
  }
},

Subscription: {
  postLiked: {
    subscribe: withFilter(
      () => pubsub.asyncIterator(['POST_LIKED']),
      (payload, variables) => {
        return payload.postId === variables.postId;
      }
    )
  }
}

```

```

    },

    commentAdded: {
      subscribe: withFilter(
        () => pubsub.asyncIterator(['COMMENT_ADDED']),
        (payload, variables) => {
          return payload.comment.postId === variables.postId;
        }
      )
    },

    notificationReceived: {
      subscribe: withFilter(
        () => pubsub.asyncIterator(['NOTIFICATION_RECEIVED']),
        (payload, variables, { user }) => {
          return payload.notification.recipientId === user.id;
        }
      )
    }
  },

  // Field resolvers
  User: {
    posts: async (user, { pagination }, { dataloaders }) => {
      return await postResolvers.getUserPosts(user.id, pagination, dataloaders);
    },

    followers: async (user, { pagination }, { dataloaders }) => {
      return await userResolvers.getUserFollowers(user.id, pagination,
dataloaders);
    },

    following: async (user, { pagination }, { dataloaders }) => {
      return await userResolvers.getUserFollowing(user.id, pagination,
dataloaders);
    },

    isFollowing: async (user, args, { user: currentUser, dataloaders }) => {
      if (!currentUser) return false;
      return await userResolvers.isFollowing(currentUser.id, user.id,
dataloaders);
    }
  },

  Post: {
    author: async (post, args, { dataloaders }) => {
      return await dataloaders.userLoader.load(post.authorId);
    },
  },

```

```

category: async (post, args, { dataloaders }) => {
  if (!post.categoryId) return null;
  return await dataloaders.categoryLoader.load(post.categoryId);
},

comments: async (post, { pagination }, { dataloaders }) => {
  return await commentResolvers.getPostComments(post.id, pagination,
dataloaders);
},

likes: async (post, args, { dataloaders }) => {
  return await dataloaders.postLikesLoader.load(post.id);
},

isLiked: async (post, args, { user, dataloaders }) => {
  if (!user) return false;
  return await postResolvers.isPostLikedByUser(post.id, user.id, dataloaders);
}
},

Comment: {
  author: async (comment, args, { dataloaders }) => {
    return await dataloaders.userLoader.load(comment.authorId);
  },

  post: async (comment, args, { dataloaders }) => {
    return await dataloaders.postLoader.load(comment.postId);
  },

  replies: async (comment, args, { dataloaders }) => {
    return await commentResolvers.getCommentReplies(comment.id, dataloaders);
  }
}
};

module.exports = resolvers;

```

4. DataLoaders for Efficient Database Queries

```

// services/graphql-service/src/dataloaders/index.js
const DataLoader = require('dataloader');
const User = require('../models/User.model');
const Post = require('../models/Post.model');
const Comment = require('../models/Comment.model');
const Category = require('../models/Category.model');

const createDataLoaders = () => {

```

 Copy

```

return {
  // User loader
  userLoader: new DataLoader(async (userIds) => {
    const users = await User.find({ _id: { $in: userIds } });
    return userIds.map(id => users.find(user => user._id.toString() === id));
  }),

  // Post loader
  postLoader: new DataLoader(async (postIds) => {
    const posts = await Post.find({ _id: { $in: postIds } });
    return postIds.map(id => posts.find(post => post._id.toString() === id));
  }),

  // Category loader
  categoryLoader: new DataLoader(async (categoryIds) => {
    const categories = await Category.find({ _id: { $in: categoryIds } });
    return categoryIds.map(id => categories.find(category =>
category._id.toString() === id));
  }),

  // Post likes loader
  postLikesLoader: new DataLoader(async (postIds) => {
    const likes = await Like.find({ postId: { $in: postIds } })
      .populate('user', 'name avatar');

    return postIds.map(postId =>
      likes.filter(like => like.postId.toString() === postId)
    );
  }),

  // User posts count loader
  userPostsCountLoader: new DataLoader(async (userIds) => {
    const counts = await Post.aggregate([
      { $match: { authorId: { $in: userIds } } },
      { $group: { _id: '$authorId', count: { $sum: 1 } } }
    ]);

    return userIds.map(userId => {
      const count = counts.find(c => c._id.toString() === userId);
      return count ? count.count : 0;
    });
  }),

  // User followers count loader
  userFollowersCountLoader: new DataLoader(async (userIds) => {
    const Follow = require('../models/Follow.model');
    const counts = await Follow.aggregate([
      { $match: { followedId: { $in: userIds } } },
      { $group: { _id: '$followedId', count: { $sum: 1 } } }
    ]);
  })
}

```

```

    });

    return userIds.map(userId => {
      const count = counts.find(c => c._id.toString() === userId);
      return count ? count.count : 0;
    });
  },

  // Post comments count loader
  postCommentsCountLoader: new DataLoader(async (postIds) => {
    const counts = await Comment.aggregate([
      { $match: { postId: { $in: postIds } } },
      { $group: { _id: '$postId', count: { $sum: 1 } } }
    ]);

    return postIds.map(postId => {
      const count = counts.find(c => c._id.toString() === postId);
      return count ? count.count : 0;
    });
  })
};

module.exports = { createDataLoaders };

```

Advanced Search with Elasticsearch

1. Elasticsearch Setup

```
npm install @elastic/elasticsearch
```

 Copy

```

// services/search-service/src/config/elasticsearch.js
const { Client } = require('@elastic/elasticsearch');
const logger = require('../../../../shared/utils/logger');

class ElasticsearchService {
  constructor() {
    this.client = new Client({
      node: process.env.ELASTICSEARCH_URL || 'http://localhost:9200',
      auth: {
        username: process.env.ELASTICSEARCH_USERNAME,
        password: process.env.ELASTICSEARCH_PASSWORD
      },
    });
  }
}

```

 Copy

```
    requestTimeout: 30000,
    pingTimeout: 3000,
    sniffOnStart: true
  });

  this.indices = {
    posts: 'posts',
    users: 'users',
    categories: 'categories'
  };

  this.initializeIndices();
}

async initializeIndices() {
  try {
    // Create posts index
    await this.createIndex(this.indices.posts, {
      mappings: {
        properties: {
          title: {
            type: 'text',
            analyzer: 'standard',
            fields: {
              keyword: { type: 'keyword' },
              suggest: { type: 'completion' }
            }
          },
          content: {
            type: 'text',
            analyzer: 'standard'
          },
          excerpt: {
            type: 'text',
            analyzer: 'standard'
          },
          tags: {
            type: 'keyword'
          },
          authorId: { type: 'keyword' },
          authorName: {
            type: 'text',
            fields: { keyword: { type: 'keyword' } }
          },
          categoryId: { type: 'keyword' },
          categoryName: {
            type: 'text',
            fields: { keyword: { type: 'keyword' } }
          }
        }
      }
    });
  } catch (error) {
    console.error('Error creating index:', error);
  }
}
```

```

    status: { type: 'keyword' },
    visibility: { type: 'keyword' },
    likesCount: { type: 'integer' },
    commentsCount: { type: 'integer' },
    viewsCount: { type: 'integer' },
    readTime: { type: 'integer' },
    publishedAt: { type: 'date' },
    createdAt: { type: 'date' },
    updatedAt: { type: 'date' },
    location: { type: 'geo_point' },
    embedding: {
      type: 'dense_vector',
      dims: 1536 // OpenAI embedding dimensions
    }
  },
},
settings: {
  analysis: {
    analyzer: {
      content_analyzer: {
        type: 'custom',
        tokenizer: 'standard',
        filter: ['lowercase', 'stop', 'snowball']
      }
    }
  }
}
});

```

```

// Create users index
await this.createIndex(this.indices.users, {
  mappings: {
    properties: {
      name: {
        type: 'text',
        analyzer: 'standard',
        fields: {
          keyword: { type: 'keyword' },
          suggest: { type: 'completion' }
        }
      },
      email: { type: 'keyword' },
      bio: { type: 'text' },
      location: {
        type: 'text',
        fields: { keyword: { type: 'keyword' } }
      },
      website: { type: 'keyword' },
      role: { type: 'keyword' },

```

```

        verified: { type: 'boolean' },
        followersCount: { type: 'integer' },
        followingCount: { type: 'integer' },
        postsCount: { type: 'integer' },
        createdAt: { type: 'date' },
        lastLogin: { type: 'date' }
    }
}
});

logger.info('Elasticsearch indices initialized successfully');
} catch (error) {
    logger.error('Failed to initialize Elasticsearch indices:', error);
}
}

async createIndex(indexName, config) {
    try {
        const exists = await this.client.indices.exists({ index: indexName });

        if (!exists) {
            await this.client.indices.create({
                index: indexName,
                body: config
            });
            logger.info(`Created Elasticsearch index: ${indexName}`);
        }
    } catch (error) {
        logger.error(`Failed to create index ${indexName}:`, error);
        throw error;
    }
}

// Index a document
async indexDocument(index, id, document) {
    try {
        await this.client.index({
            index,
            id,
            body: document,
            refresh: 'wait_for'
        });

        logger.debug(`Indexed document ${id} in ${index}`);
    } catch (error) {
        logger.error(`Failed to index document ${id}:`, error);
        throw error;
    }
}
}

```

```
// Update a document
async updateDocument(index, id, document) {
  try {
    await this.client.update({
      index,
      id,
      body: { doc: document },
      refresh: 'wait_for'
    });

    logger.debug(`Updated document ${id} in ${index}`);
  } catch (error) {
    logger.error(`Failed to update document ${id}:`, error);
    throw error;
  }
}

// Delete a document
async deleteDocument(index, id) {
  try {
    await this.client.delete({
      index,
      id,
      refresh: 'wait_for'
    });

    logger.debug(`Deleted document ${id} from ${index}`);
  } catch (error) {
    logger.error(`Failed to delete document ${id}:`, error);
    throw error;
  }
}

// Multi-index search
async search(query, options = {}) {
  try {
    const searchBody = this.buildSearchQuery(query, options);

    const response = await this.client.search({
      index: options.indices || Object.values(this.indices),
      body: searchBody,
      size: options.size || 20,
      from: options.from || 0
    });

    return this.formatSearchResults(response);
  } catch (error) {
    logger.error('Search failed:', error);
  }
}
```

```
        throw error;
    }
}

buildSearchQuery(query, options) {
    const must = [];
    const filter = [];

    // Main search query
    if (query) {
        must.push({
            multi_match: {
                query,
                fields: [
                    'title^3',
                    'content^2',
                    'name^3',
                    'bio',
                    'tags^2',
                    'authorName',
                    'categoryName'
                ],
                type: 'best_fields',
                fuzziness: 'AUTO',
                prefix_length: 2
            }
        });
    }

    // Filters
    if (options.filters) {
        Object.entries(options.filters).forEach(([key, value]) => {
            if (value !== undefined && value !== null) {
                if (Array.isArray(value)) {
                    filter.push({ terms: { [key]: value } });
                } else {
                    filter.push({ term: { [key]: value } });
                }
            }
        });
    }

    // Date range filter
    if (options.dateRange) {
        filter.push({
            range: {
                [options.dateField || 'createdAt']: {
                    gte: options.dateRange.from,
                    lte: options.dateRange.to
                }
            }
        });
    }
}
```

```
    }
  }
});
}

// Geographic search
if (options.location && options.distance) {
  filter.push({
    geo_distance: {
      distance: options.distance,
      location: options.location
    }
  });
}

const searchQuery = {
  query: {
    bool: {
      must,
      filter
    }
  }
};

// Sorting
if (options.sort) {
  searchQuery.sort = options.sort;
} else {
  searchQuery.sort = [
    { _score: { order: 'desc' } },
    { createdAt: { order: 'desc' } }
  ];
}

// Highlighting
searchQuery.highlight = {
  fields: {
    title: {},
    content: { fragment_size: 150, number_of_fragments: 3 },
    name: {},
    bio: { fragment_size: 100, number_of_fragments: 1 }
  }
};

// Aggregations for faceted search
if (options.aggregations) {
  searchQuery.aggs = {
    categories: {
      terms: { field: 'categoryName.keyword', size: 10 }
    }
  }
}
```

```

    },
    tags: {
      terms: { field: 'tags', size: 20 }
    },
    authors: {
      terms: { field: 'authorName.keyword', size: 10 }
    },
    date_histogram: {
      date_histogram: {
        field: 'createdAt',
        calendar_interval: 'month'
      }
    }
  };
}

return searchQuery;
}

formatSearchResults(response) {
  return {
    total: response.hits.total.value,
    results: response.hits.hits.map(hit => ({
      id: hit._id,
      index: hit._index,
      score: hit._score,
      source: hit._source,
      highlights: hit.highlight || {}
    })),
    aggregations: response.aggregations || {},
    took: response.took
  };
}

// Autocomplete suggestions
async suggest(query, field = 'title.suggest', size = 10) {
  try {
    const response = await this.client.search({
      index: Object.values(this.indices),
      body: {
        suggest: {
          autocomplete: {
            prefix: query,
            completion: {
              field,
              size
            }
          }
        }
      }
    })
  }
}

```

```

    }
  });

  return response.suggest.autocomplete[0].options.map(option => ({
    text: option.text,
    score: option._score,
    source: option._source
  }));
} catch (error) {
  logger.error('Autocomplete failed:', error);
  throw error;
}
}

// Semantic search using embeddings
async semanticSearch(queryEmbedding, options = {}) {
  try {
    const response = await this.client.search({
      index: this.indices.posts,
      body: {
        query: {
          script_score: {
            query: {
              bool: {
                filter: options.filters || []
              }
            },
            script: {
              source: "cosineSimilarity(params.query_vector, 'embedding') +
1.0",
              params: {
                query_vector: queryEmbedding
              }
            }
          }
        },
        size: options.size || 10
      }
    });

    return this.formatSearchResults(response);
  } catch (error) {
    logger.error('Semantic search failed:', error);
    throw error;
  }
}

// More Like This query
async findSimilar(documentId, index, options = {}) {

```

```
try {
  const response = await this.client.search({
    index,
    body: {
      query: {
        more_like_this: {
          fields: ['title', 'content', 'tags'],
          like: [
            {
              _index: index,
              _id: documentId
            }
          ],
          min_term_freq: 2,
          max_query_terms: 12,
          minimum_should_match: '30%'
        }
      },
      size: options.size || 5
    }
  });

  return this.formatSearchResults(response);
} catch (error) {
  logger.error('Similar documents search failed:', error);
  throw error;
}

// Analytics and metrics
async getSearchAnalytics(options = {}) {
  try {
    const response = await this.client.search({
      index: '_all',
      body: {
        size: 0,
        aggs: {
          indices_stats: {
            terms: { field: '_index' }
          },
          daily_docs: {
            date_histogram: {
              field: 'createdAt',
              calendar_interval: 'day',
              min_doc_count: 1
            }
          },
          top_tags: {
            terms: {
```

```

        field: 'tags',
        size: 20
      }
    }
  }
}
});

return {
  totalDocuments: response.hits.total.value,
  indicesStats: response.aggregations.indices_stats.buckets,
  dailyStats: response.aggregations.daily_docs.buckets,
  topTags: response.aggregations.top_tags.buckets
};
} catch (error) {
  logger.error('Search analytics failed:', error);
  throw error;
}
}
}

module.exports = new ElasticsearchService();

```

2. Search Service Implementation

```

// services/search-service/src/services/search.service.js
const elasticsearchService = require('../config/elasticsearch');
const aiService = require('../../ai-service/src/config/openai');
const logger = require('../../../../shared/utils/logger');

class SearchService {

  // Universal search across all content types
  async universalSearch(query, options = {}) {
    try {
      const searchOptions = {
        indices: options.indices,
        size: options.limit || 20,
        from: ((options.page || 1) - 1) * (options.limit || 20),
        filters: this.buildFilters(options),
        sort: this.buildSort(options),
        aggregations: true
      };

      // Perform main search
      const searchResults = await elasticsearchService.search(query,
searchOptions);

```

 Copy

```

// If semantic search is enabled and we have AI service
let semanticResults = null;
if (options.semantic && query) {
  try {
    const embedding = await aiService.generateEmbeddings(query);
    semanticResults = await elasticsearchService.semanticSearch(
      embedding.embeddings[0].embedding,
      { size: 5, filters: searchOptions.filters }
    );
  } catch (error) {
    logger.warn('Semantic search failed, falling back to text search:',
error);
  }
}

return {
  query,
  results: searchResults.results,
  semanticResults: semanticResults?.results || [],
  facets: this.processFacets(searchResults.aggregations),
  pagination: {
    page: options.page || 1,
    limit: options.limit || 20,
    total: searchResults.total,
    pages: Math.ceil(searchResults.total / (options.limit || 20))
  },
  took: searchResults.took
};
} catch (error) {
  logger.error('Universal search failed:', error);
  throw new Error('Search service unavailable');
}
}

// Autocomplete and suggestions
async autocomplete(query, options = {}) {
  try {
    const suggestions = await elasticsearchService.suggest(
      query,
      options.field || 'title.suggest',
      options.size || 10
    );

    // Get search history for personalized suggestions
    const popularSearches = await this.getPopularSearches(query, 5);

    return {
      suggestions: suggestions.map(s => ({

```

```

        text: s.text,
        score: s.score,
        type: this.determineResultType(s.source),
        source: s.source
    })),
    popular: popularSearches,
    query
};
} catch (error) {
    logger.error('Autocomplete failed:', error);
    return { suggestions: [], popular: [], query };
}
}

// Advanced filtering and faceted search
async facetedSearch(query, facets = {}) {
    try {
        const filters = {};

        // Convert facets to Elasticsearch filters
        Object.entries(facets).forEach(([key, values]) => {
            if (values && values.length > 0) {
                filters[key] = Array.isArray(values) ? values : [values];
            }
        });

        const searchResults = await elasticsearchService.search(query, {
            filters,
            aggregations: true,
            size: 50
        });

        return {
            results: searchResults.results,
            facets: this.processFacets(searchResults.aggregations),
            applied_filters: facets,
            total: searchResults.total
        };
    } catch (error) {
        logger.error('Faceted search failed:', error);
        throw error;
    }
}

// Geographic search
async geoSearch(query, location, distance = '10km', options = {}) {
    try {
        const searchOptions = {
            location: {

```

```
        lat: location.lat,
        lon: location.lon
    },
    distance,
    filters: options.filters,
    size: options.limit || 20
};

const results = await elasticsearchService.search(query, searchOptions);

return {
    results: results.results.map(result => ({
        ...result,
        distance: this.calculateDistance(
            location,
            result.source.location
        )
    })),
    center: location,
    radius: distance,
    total: results.total
};
} catch (error) {
    logger.error('Geo search failed:', error);
    throw error;
}
}

// Content recommendation based on user behavior
async getRecommendations(userId, options = {}) {
    try {
        // Get user's interaction history
        const userHistory = await this.getUserSearchHistory(userId);
        const userInteractions = await this.getUserInteractions(userId);

        // Build recommendation query based on user preferences
        const recommendations = [];

        // Content-based recommendations
        if (userInteractions.likedPosts.length > 0) {
            const contentRecs = await this.getContentBasedRecommendations(
                userInteractions.likedPosts.slice(0, 5)
            );
            recommendations.push(...contentRecs.results);
        }

        // Collaborative filtering recommendations
        const collaborativeRecs = await
this.getCollaborativeRecommendations(userId);
```

```
recommendations.push(...collaborativeRecs);

// Trending content
const trending = await this.getTrendingContent();
recommendations.push(...trending);

// Remove duplicates and limit results
const uniqueRecs = this.removeDuplicates(recommendations);

return {
  recommendations: uniqueRecs.slice(0, options.limit || 10),
  categories: {
    contentBased: contentRecs?.results.length || 0,
    collaborative: collaborativeRecs.length,
    trending: trending.length
  }
};
} catch (error) {
  logger.error('Recommendations failed:', error);
  return { recommendations: [], categories: {} };
}
}

// Real-time search suggestions with typo tolerance
async smartSearch(query, options = {}) {
  try {
    // Check for typos and suggest corrections
    const spellcheck = await this.checkSpelling(query);

    let searchQuery = query;
    let suggestions = [];

    if (spellcheck.hasSuggestions) {
      // Search with corrected query
      const correctedResults = await elasticsearchService.search(
        spellcheck.corrected,
        { size: 5 }
      );

      if (correctedResults.total > 0) {
        suggestions = [{
          type: 'spelling',
          original: query,
          corrected: spellcheck.corrected,
          results: correctedResults.results
        }];
      }
    }
  }
}
```

```
// Main search
const mainResults = await elasticsearchService.search(searchQuery, {
  size: options.limit || 20,
  from: ((options.page || 1) - 1) * (options.limit || 20)
});

// Similar searches
const similarSearches = await this.findSimilarSearches(query);

return {
  results: mainResults.results,
  suggestions,
  similar: similarSearches,
  corrected: spellcheck.hasSuggestions ? spellcheck.corrected : null,
  total: mainResults.total
};
} catch (error) {
  logger.error('Smart search failed:', error);
  throw error;
}
}

// Search analytics and insights
async getSearchAnalytics(options = {}) {
  try {
    const analytics = await elasticsearchService.getSearchAnalytics(options);

    // Get popular searches
    const popularSearches = await this.getPopularSearches('', 20);

    // Get search trends
    const trends = await this.getSearchTrends(options.timeframe || '30d');

    return {
      overview: {
        totalDocuments: analytics.totalDocuments,
        indicesCount: analytics.indicesStats.length,
        searchesPerDay: trends.searchesPerDay
      },
      popular: popularSearches,
      trends: trends.trends,
      performance: {
        averageResponseTime: trends.averageResponseTime,
        slowQueries: trends.slowQueries
      },
      content: {
        topTags: analytics.topTags,
        contentDistribution: analytics.indicesStats
      }
    }
  }
}
```

```
};  
} catch (error) {  
  logger.error('Search analytics failed:', error);  
  throw error;  
}  
}  
  
// Helper methods  
buildFilters(options) {  
  const filters = [];  
  
  if (options.contentType) {  
    filters.push({ term: { _index: options.contentType } });  
  }  
  
  if (options.author) {  
    filters.push({ term: { 'authorId': options.author } });  
  }  
  
  if (options.category) {  
    filters.push({ term: { 'categoryId': options.category } });  
  }  
  
  if (options.tags && options.tags.length > 0) {  
    filters.push({ terms: { 'tags': options.tags } });  
  }  
  
  if (options.dateRange) {  
    filters.push({  
      range: {  
        createdAt: {  
          gte: options.dateRange.from,  
          lte: options.dateRange.to  
        }  
      }  
    });  
  }  
  
  if (options.status) {  
    filters.push({ term: { status: options.status } });  
  }  
  
  return filters;  
}  
  
buildSort(options) {  
  const sortMap = {  
    relevance: [{ _score: { order: 'desc' } }],  
    date: [{ createdAt: { order: 'desc' } }],  
  };  
}
```

```

    popularity: [{ likesCount: { order: 'desc' } }, { viewsCount: { order:
'desc' } }],
    alphabetical: [{ 'title.keyword': { order: 'asc' } }]
  };

  return sortMap[options.sort] || sortMap.relevance;
}

processFacets(aggregations) {
  if (!aggregations) return {};

  const facets = {};

  Object.entries(aggregations).forEach(([key, agg]) => {
    if (agg.buckets) {
      facets[key] = agg.buckets.map(bucket => ({
        value: bucket.key,
        count: bucket.doc_count
      }));
    }
  });

  return facets;
}

determineResultType(source) {
  if (source.title && source.content) return 'post';
  if (source.name && source.email) return 'user';
  if (source.name && source.description) return 'category';
  return 'unknown';
}

calculateDistance(point1, point2) {
  // Haversine formula for calculating distance
  const R = 6371; // Earth's radius in kilometers
  const dLat = this.toRad(point2.lat - point1.lat);
  const dLon = this.toRad(point2.lon - point1.lon);
  const lat1 = this.toRad(point1.lat);
  const lat2 = this.toRad(point2.lat);

  const a = Math.sin(dLat/2) * Math.sin(dLat/2) +
    Math.sin(dLon/2) * Math.sin(dLon/2) * Math.cos(lat1) *
Math.cos(lat2);
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));

  return (R * c).toFixed(2) + ' km';
}

toRad(deg) {

```

```
    return deg * (Math.PI/180);
  }

  removeDuplicates(array) {
    const seen = new Set();
    return array.filter(item => {
      const id = item.id || item._id;
      if (seen.has(id)) return false;
      seen.add(id);
      return true;
    });
  }

  // Placeholder methods for additional functionality
  async getUserSearchHistory(userId) {
    // Implementation would fetch from search history database
    return { searches: [], preferences: {} };
  }

  async getUserInteractions(userId) {
    // Implementation would fetch user's likes, comments, etc.
    return { likedPosts: [], viewedPosts: [], searchHistory: [] };
  }

  async getContentBasedRecommendations(likedPosts) {
    // Use "More Like This" query with liked posts
    const recommendations = [];

    for (const postId of likedPosts) {
      try {
        const similar = await elasticsearchService.findSimilar(
          postId,
          'posts',
          { size: 3 }
        );
        recommendations.push(...similar.results);
      } catch (error) {
        logger.warn(`Failed to get similar posts for ${postId}:`, error);
      }
    }

    return { results: this.removeDuplicates(recommendations) };
  }

  async getCollaborativeRecommendations(userId) {
    // Implementation would use collaborative filtering algorithms
    return [];
  }
}
```

```
async getTrendingContent() {
  // Get trending posts based on recent engagement
  try {
    const trending = await elasticsearchService.search('', {
      filters: [
        {
          range: {
            createdAt: {
              gte: 'now-7d'
            }
          }
        }
      ],
      sort: [
        { likesCount: { order: 'desc' } },
        { commentsCount: { order: 'desc' } }
      ],
      size: 10
    });

    return trending.results;
  } catch (error) {
    logger.error('Failed to get trending content:', error);
    return [];
  }
}

async checkSpelling(query) {
  // Simple spell check implementation
  // In production, you might use a dedicated spell-check service
  return {
    hasSuggestions: false,
    corrected: query,
    suggestions: []
  };
}

async findSimilarSearches(query) {
  // Find searches with similar terms
  return [];
}

async getPopularSearches(prefix, limit) {
  // Get popular searches from analytics
  return [];
}

async getSearchTrends(timeframe) {
  // Get search trends and performance metrics
```

```
    return {
      searchesPerDay: 0,
      trends: [],
      averageResponseTime: 0,
      slowQueries: []
    };
  }
}

module.exports = new SearchService();
```

Chapter 5 Summary

Congratulations! You've mastered **cutting-edge backend technologies**:

AI/ML Integration

- **OpenAI API integration** for text generation and completion
- **Content moderation** with AI-powered analysis
- **Image generation** and audio transcription
- **Semantic search** using embeddings
- **Content creation automation**

GraphQL APIs

- **Complete GraphQL server** with Apollo Server
- **Schema-first development** with type definitions
- **Efficient data loading** with DataLoaders
- **Real-time subscriptions** for live updates
- **Query optimization** and security

Advanced Search

- **Elasticsearch integration** for powerful search
- **Semantic search** with AI embeddings
- **Faceted search** and filtering
- **Geographic search** capabilities
- **Autocomplete** and suggestions
- **Search analytics** and insights

Smart Features

- **Content recommendations** based on user behavior
- **Real-time analytics** and reporting

- **Advanced file processing** with AI
- **Multi-language support** and detection
- **Performance monitoring** and optimization

What You Can Build Now

With 5 chapters completed, you can create **industry-leading applications**:

AI-Powered Platforms

- **Content creation tools** (like Jasper, Copy.ai)
- **Smart social media platforms** with AI moderation
- **Intelligent search engines** (like Algolia)
- **Recommendation systems** (like Netflix, Amazon)
- **AI-powered analytics dashboards**

Advanced Applications

- **Multi-tenant SaaS platforms** with advanced search
- **Real-time collaboration tools** with GraphQL
- **E-commerce platforms** with AI recommendations
- **Content management systems** with AI assistance
- **Data analytics platforms** with ML insights

Enterprise Solutions

- **Customer support platforms** with AI chatbots
- **Marketing automation** with AI content generation
- **Business intelligence dashboards**
- **Advanced CRM systems** with predictive analytics

Final Chapter Preview

Chapter 6: Cloud Native & Enterprise

- **AWS/Azure/GCP deployment strategies**
- **Kubernetes orchestration** and auto-scaling
- **Advanced security patterns** and compliance
- **Enterprise integration** with legacy systems
- **Multi-region deployment** and disaster recovery
- **Advanced monitoring** and observability



Ready for the Final Chapter?

You're now at a **professional/expert level** and can build applications that compete with industry leaders! The final chapter will make you a **cloud architect** capable of deploying and scaling applications globally.

Want to complete your journey with Chapter 6: Cloud Native & Enterprise? ☁